

The Shark Tank Project

A complete, plain-English walkthrough of the daily cross-asset “Market Map” brief — the machinery (Claude Code, the terminal, GitHub, Vercel), every data source, and the exact reasoning behind every tab: narrative, earnings, trade ideas, the client portfolio (“Fable”), and the derivatives desk. Written so that someone who has never opened a terminal can follow it end to end.

Document generated 11 June 2026 · Repo: [harryimposter/shark-tank](#) · Live site: [harinidesai.com](#)

There is one idea that makes the whole project make sense, and everything else is detail hanging off it. Read this first.

The one big idea

The “intelligence” in this product is Claude doing research and writing reasoning into a Python file. The Python file itself is dumb. It has no AI, no API key, no model call. When you ask for a refresh, *Claude* (the model, in Claude Code) does the web searching, forms the macro view, picks the trades, writes the words — and bakes all of that into a dated Python “generator.” That generator then pulls a handful of *live numbers* from the internet (prices, RSI, earnings) and renders six HTML pages. So the brief is part **authored** (the judgement, done by Claude each morning) and part **computed** (the live marks and technicals, done by the code at run time).

1 The Machine — Claude Code, terminal, GitHub, Vercel

1.1 The terminal and Claude Code, for someone who has never used either

A **terminal** (also “command line” or “shell”) is a text window where you type instructions to your computer instead of clicking buttons. On this machine it’s **PowerShell** on Windows. Instead of double-clicking a program, you type its name and press Enter.

Claude Code is Anthropic’s coding tool that runs *inside* that terminal. It is Claude (the AI) given the ability to read files on the computer, write and edit files, run commands, search the web, and use external tools. You talk to it in plain English (“run the Market Map”), and it carries out multi-step work: reading the spec, searching the news, writing code, running it, and pushing the result online.

So in this project the terminal is just the room Claude Code works in. You are never typing cryptic commands yourself — you type an instruction in English, and Claude Code does the typing of real commands on your behalf.

The three things Claude Code does with the terminal here

- **Reads & writes files** — it opens `PROMPT.md` (the spec), reads yesterday's generator, and writes today's.
- **Runs Python** — it executes `python gen_2026_06_11.py`, which fetches live data and renders the HTML pages.
- **Runs git** — it commits the new files and pushes them to GitHub, which triggers the website to update.

1.2 Where the code is stored

All the code lives in one folder on the local computer:

```
C:\Users\harin\OneDrive\Desktop\Claude Code\Shark Tank\
```

That folder is a **git repository** — a folder whose change-history is tracked. It contains the Python “engine” files, the data files (JSON), the spec (`PROMPT.md`), and the generated HTML pages. The code is *not* stored “in the terminal” — the terminal is only the tool used to act on the folder. The code sits on disk (and, because the folder is under OneDrive, it's also mirrored to cloud storage as a side effect — but that's incidental, not how it deploys).

1.3 GitHub — the shared copy of the folder

GitHub is a website that hosts a copy of that folder in the cloud (the “remote,” named `origin`, at `github.com/harryimposter/shark-tank`). Two things happen there:

- **It's the source of truth the website reads from.** When Claude “pushes” the new files to GitHub's `main` branch, that's what makes the website update.
- **It runs two scheduled robots (“GitHub Actions”)** on its own servers — independent of your computer:
 - `earnings-data.yml` — every weekday ~6am UTC it runs `fetch_earnings.py` against Finnhub and commits a fresh `earnings_data.md`. (This is why a morning refresh push sometimes has to `git rebase` first — the robot already committed.)
 - `fallback-refresh.yml` — a safety net (see §1.6).

1.4 Vercel — how the HTML becomes a live website

Vercel (Netlify is configured as an equivalent backup) is a hosting service wired to the GitHub repo. The arrangement is the simplest possible: this is a *static site*. There is no build step, no server, no database.

- `vercel.json` says: `buildCommand: null`, `outputDirectory: "."` — i.e. “don't build anything; just serve the files in the folder as-is.”
- `.vercelignore` excludes the Python files, `requirements.txt`, `run.sh` and `PROMPT.md` — so Vercel doesn't mistake the generators for something to run. **The Python never executes on Vercel.** Only the finished HTML is served.

- `index.html` is the home page (the Summary tab); the other tabs are plain files at `/insights.html`, `/earnings.html`, `/trades.html`, `/portfolio.html`, `/ideas.html`.

The chain is: Claude writes HTML → push to GitHub `main` → Vercel detects the push → Vercel republishes the folder → harinidesai.com shows the new pages. This takes a few seconds and needs no action from you.

1.5 Why we “amend the code” every single refresh

This is the question that trips everyone up, and the answer follows straight from the one big idea.

The rendered Python has **no AI inside it**. So the day’s judgement — the regime call, the wording, which four trades, the bull/base/bear, the deep-dive opinions — has to be *written down somewhere* the dumb renderer can read. That “somewhere” is a brand-new dated generator file, e.g. `gen_2026_06_10.py`, which contains a big Python dictionary called `brief` holding all of today’s prose and structured views.

So “amending the code” is really **Claude authoring today’s thinking into a new file**. It is not editing a config — it is the act of writing the brief. Compare the two halves:

AUTHORED FRESH EACH DAY (BY CLAUDE)	COMPUTED AT RUN TIME (BY THE CODE)
Regime name & the 800–1000 word wrap	Live index/FX/rates/commodity levels
The 4 trade ideas + theses	RSI, moving averages, Bollinger, Fibonacci
Bull/Base/Bear & probabilities	The ~140-name RSI screener
Deep-dive opinions, catalyst calendar	Client-book marks & P&L
House views per holding, derivatives ideas	Implied/realized vol (Black–Scholes inversion)

A new file is used (rather than editing yesterday’s) so the history is honest and auditable: every day’s call is frozen in its own file, and `trades.json` can grade past calls against what actually happened. The spec is explicit about this — “build on yesterday, don’t start cold”: today’s generator is literally written by reading yesterday’s.

1.6 Where it actually runs when we refresh

There are three execution paths, in order of preference:

(a) **You, on demand.** You say “run the Market Map.” Claude Code, on this Windows machine, does the full real refresh — web sweep, live marks, book scan — writes the pages and pushes. This is the gold-standard path.

(b) **The local scheduled agent** — `market-map-daily-refresh`, cron `0 6 * * 1-5` (~6am London, weekdays). It does the same genuine full refresh and auto-pushes, but only **while the Claude app is open** on the machine (it’s host-dependent, not a cloud server). It’s gated by sanity checks: it aborts the push if the generator errors, if zero symbols/zero screener names come back, if `trades.html` renders under 20 KB, or if any page is missing.

(c) **The GitHub fallback** — `fallback-refresh.yml`, 07:30 UTC weekdays, runs on GitHub’s servers. It first checks whether the local agent already wrote today’s `gen_2026_<today>.py`; if so it *skips*. If not (laptop was closed), it re-runs the latest generator in **data-only mode** (`MARKETMAP_REFRESH_MODE=fallback`): live marks/RSI/screener/book all refresh, but the *narrative carries over* from the last full day. In that mode the Summary page shows an amber “data-only fallback refresh” banner and the coverage bar drops the “web-sourced” claim and cites the carried date. It is sanity-gated the same way.

The key honesty point baked into the design: **live numbers are always real**, fetched at run time; the *narrative* is only as fresh as the last genuine web sweep, and the UI says which it is.

1.7 The full refresh pipeline, step by step

When the generator runs (`python gen_2026_06_11.py`), this is the exact sequence:

- 1 Load state**
Read `trades.json` (the open/closed trade book) and `regime_log.json` (the dated history of regime banners).
- 2 Fetch live levels**
`live_levels.fetch()` hits TradingView’s public scanner endpoint for ~24 macro instruments (S&P, Nasdaq, DAX, Nikkei, FTSE, EURUSD, USDJPY, DXY, US/Bund/Gilt yields, WTI, Brent, gold, VIX...). Derived spreads (Brent–WTI, 2s10s, DAX/Nasdaq) are computed here.
- 3 Mark the book to market**
Each open trade’s live level is mapped in; P&L% is recomputed respecting direction (shorts profit when price falls); a dated row is appended to that trade’s history. Stops/targets are checked — but never closed before `min_hold_days`.

4

RSI + technicals

`fetch_rsi.fetch_all()` pulls ~1 year of daily closes from Yahoo Finance for each open trade and each derivatives idea, computes 14-period Wilder RSI on the absolute 30/70 convention, plus 50/100/200-day MAs, 20-day Bollinger Bands, and Fibonacci retracements off the 6-month swing.

5

RSI screener

`fetch_rsi.run_screener()` scans the curated ~140-name cross-asset universe, computes RSI for each, and splits out the oversold (≤ 30) and overbought (≥ 70) names.

6

Client-book scan

`book_scanner.build_scan(brief)` marks Fable's portfolio to live levels, attaches the per-name house views and the implied-vol snapshot, and builds the ranked derivatives-desk shelf and loss-recovery routes.

7

Render

`book.build_html(...)` writes the legacy `output.html`; `shark_format.render_all(brief, trades, regime_log, scan=scan)` writes the six tab pages
(`index/insights/earnings/trades/portfolio/ideas.html`) plus fragments in `frag/`.

8

Persist & deploy

`trades.json` and `regime_log.json` are saved with today's marks. Then `git add -A && git commit && git push` → Vercel redeploys.

2 Data sources & the “TradingView MCP” question

2.1 What MCP actually is

MCP = Model Context Protocol. It’s an open standard (from Anthropic) for plugging external tools and data sources into an AI like Claude in a uniform way. Think of it as a universal adapter: instead of hand-coding a connection to each service, an “MCP server” advertises a set of tools (e.g. “get_quote”, “search_filings”), and Claude can call them as if they were built-in. In Claude Code you add one with a command like `claude mcp add ...`.

Important correction: this project does *not* use a “TradingView MCP”

It’s worth being precise, because the design philosophy of the whole project is honesty about sources.

There is no TradingView MCP server wired into the live product. The settings show one experiment —

`claude mcp add --transport http financial-datasets ...` — but the data path that actually runs every day is plain HTTP calls from standard-library Python. “TradingView” appears in the code and the page footers because of two real but non-MCP things, explained next.

2.2 Every data source, and exactly what each one feeds

SOURCE	HOW IT'S ACCESSED	WHAT IT FEEDS	CONFIDENCE
TradingView scanner <code>scanner.tradingview.com/global/scan</code>	<code>live_levels.py</code> , direct <code>urllib</code> POST, no key, no browser	Live macro dashboard levels; live marks for the trade book & the client portfolio's liquid holdings	Live (non-US lines can be ~15-min delayed)
Yahoo Finance <code>query1.finance.yahoo.com</code>	<code>fetch_rsi.py</code> (chart API) & <code>fetch_ivol.py</code> (options chain, cookie+crumb)	RSI, 50/100/200 MAs, Bollinger, Fibonacci, the ~140-name screener, P/E & EV/EBITDA, and implied/realized vol	Sourced (EOD/1-yr history)
Finnhub <code>finnhub.io</code>	<code>fetch_earnings.py</code> , run by a GitHub Action at 6am, writes <code>earnings_data.md</code>	Earnings calendar, consensus EPS, analyst recommendations, surprise history, growth metrics	Sourced (API key, server-side)
Web search (Claude's tool)	Claude runs ≥16 searches across an allowlist (<code>sources.py</code>)	The entire narrative: overnight events, geopolitics, central banks, single-name surprises, the political read	Authored from cited sources
stooq.com	Allowed in the spec as a price fallback	Backup live levels where TradingView doesn't resolve	Live/EOD

The **source allowlist** in `sources.py` is the discipline behind the narrative: wires (Reuters, Bloomberg, FT, WSJ, AP), officials (Fed, ECB, BoE, BoJ, BLS, BEA, Treasury, CFTC, IMF, OECD), exchanges (CME, Cboe, ICE), and named independents (Doomberg, Apollo/Slok, Fed Guy). Anything that appears *only* off-list is dropped; primary sources are cited by name inline, anything else needs a full citation in the page's citations block.

2.3 The “TradingView” feed, explained honestly

“TradingView” shows up in three distinct ways — only the first is a data feed:

- 1. The scanner JSON endpoint (real data).** `live_levels.py` POSTs a list of tickers (e.g. `ICEEUR:BRN1!` for Brent, `TVC:VIX`, `SP:SPX`) to TradingView's public scanner and gets back `close`, `change`, `change_abs`. This is the same data behind TradingView's own widgets, read “headlessly.” It's an *unofficial* endpoint — robust today, but it could change, which is why the code falls back to file marks if the call fails.
- 2. The embedded quote widget (display only).** Every tab embeds TradingView's official `embed-widget-market-quotes.js` strip. That's a cosmetic live-quotes ribbon rendered in the visitor's browser; it carries no data into the brief.
- 3. The label “live levels via TradingView.”** The page footers and a few house-view notes say marks are “via TradingView” — accurate, because path (1) is the source. (One house-view note even tags a refreshed LVMH mark “Live mark (TradingView).”)

So, mapping your mental model to reality

When you've said "TradingView MCP," the thing doing that job is `live_levels.py` calling the TradingView scanner directly. It behaves like an MCP would (gives Claude live market levels) but it's a hard-coded HTTP client, not the MCP protocol. If we ever wanted a *true* MCP, we'd register a server and let Claude call it as a tool — and the natural candidate is the `financial-datasets` one already trialled in settings.

2.4 What we scan for — the 24-hour global sweep

Before a single word is written, the spec mandates a rolling **NOW – 24h** sweep (not a hardcoded date) with a hard floor of **≥16 web searches**, including at least one dedicated search per region, one geopolitical scan, and one single-name-surprise scan. The sweep must cover, every time:

Regions — all five

- Asia (China/HK + the rest)
- Japan
- US
- Europe (euro area + DAX/CAC/periphery)
- UK

One pass per region for its overnight session — what moved, why, and the local political angle. A move one region has made that the others haven't priced *is the trade*.

Asset classes — all of them

- Equities (indices + leaders/laggards)
- FX
- Rates (front end, long end, curve)
- Credit (IG/HY spreads)
- Commodities (oil, gas, metals, gold)
- Vol / skew, funding / plumbing
- Crypto if it's signalling

Four scans that are called out as non-negotiable, each with a "concrete lesson" attached so a past miss never repeats:

- **Geopolitics** — military action, ceasefires, sanctions, elections, chokepoints (Hormuz/Suez/Taiwan). *Lesson: never again miss a US–Iran-strikes-type event.* If it broke in the window, it leads the brief.
- **Headline / political newsflow** — Trump White House remarks, press-gaggle quotes, Truth Social posts, tariffs, executive orders, central-bank jawboning. These whipsaw the tape *intraday*, between scheduled events. *Lesson: the Jun-10 “we’re going to be attacking them very hard” line that whipsawed oil and equities.* Name the quote, the reaction, and the non-consensus read.
- **Single-name / corporate surprises** — earnings, guidance, M&A, supplier deals, CEO commentary. *Lesson: a single Jensen Huang sentence (Nvidia using SK Hynix chips) can move a whole supply chain.*
- **Macro data + central banks** — every print and every official comment from Fed/ECB/BoE/BoJ/PBoC in the window.

3 The narrative — Summary & Insights

The Summary tab is the “overnight read”; the Insights tab is the detailed map written as seamless prose. Both are authored by Claude during the refresh. Here is exactly how.

3.1 The voice DNA you set

The voice is codified in `PROMPT.md` as a strategist who has “done this job for 15 years, has a view on everything, is never neutral, writes to be right, not to be safe.” The mechanical rules:

- **Lead every section with a claim, not a status update.** Not “markets fell” → “the dollar is telling a different story to equities and one of them is wrong.”
- **Banned words**, stripped entirely: *could, may, potentially, appears to, seems to, suggests, somewhat, relatively, broadly, amid, against the backdrop of, in the wake of, market participants, investors are watching*. If a sentence needs one, it isn’t ready.
- Declarative, short sentences — then occasionally one long synthesising sentence tying three things together across asset classes.
- **Name names** and levels. Never “watch X” without a specific level or threshold. Never an observation without a directional take.
- One analogy per output, maximum. No adverbs. No “in my view” (if you wrote it, you think it).

And the seven “moves” — analytical patterns borrowed by name, while never copying anyone’s actual prose:

MOVE	WHAT IT DOES	WHERE IT LANDS
Campbell	Decompose the headline number — show the anatomy that contradicts it. Always end on: so what, who’s wrong, what’s the trade.	The wrap; correlation breaks
Doomberg pivot	Name the second-order effect consensus is missing, in one sentence.	Opening of the wrap, once
Pozsar mechanic	Trace a move back to a balance sheet, a flow, a funding constraint. Plumbing beats narrative.	Funding & plumbing, once
Papic constraint	Name the political constraint behind the macro move; give the non-consensus read.	Every event with a political angle
Perkins regime-call	Name the regime and commit — “late-cycle stagflation,” “liquidity melt-up.”	Top of the brief; the gold pill
Burry tell	The one structural thing nobody is watching that matters in 6 months. No trade attached.	“What the tape is missing,” once
Klein discipline	Every datum has a timestamp and a source; stale data is flagged.	Staleness check, throughout

3.2 The 5-layer map (how the detailed read is structured)

The wrap is built on a fixed 5-layer skeleton. On the Insights tab these are woven into seamless prose (no “Layer 1/2/3” labels); on the engine side they’re still the scaffold:

1. **L1 — the dominant driver.** The single thing explaining 60–70% of cross-asset moves today. Name the Perkins regime here.
2. **L2 — the counter-intuitive hook.** What consensus expected vs what actually happened.
3. **L3 — the gap.** Three buckets: (A) real-economy ground truth, (B) what’s priced, (C) the consensus narrative. The gap between them is where alpha lives — name it.
4. **L4 — Bull / Base / Bear** (see next).
5. **L5 — priced vs not-priced map.** 3–4 items on a spectrum from “mispriced the wrong way” to “fully priced.”

3.3 Bull / Base / Bear and how probabilities are assigned

Three scenarios rendered as green/gold/red tiles, with **probabilities that must sum to 100%**. Each scenario must answer the same four questions so they’re comparable: what happens to *risk assets / rates / FX / commodities*.

The probabilities are a **judgement call by Claude**, disciplined — not a formula — by:

- **What’s already priced** (L5). If the market has fully priced an outcome, that outcome’s marginal probability of *moving* things is low; the mispriced tail gets more weight.
- **The positioning read** — if the crowd is leaned hard one way, the “pain trade” (the other way) gets a fatter tail than naïve odds suggest.
- **Catalyst proximity** — a binary event inside the window (a CPI print, an FOMC, a ceasefire deadline) widens the distribution and shifts weight toward the scenario that event resolves.
- **Consistency with the named regime** — the base case has to be the regime continuing; bull/bear are the two ways it breaks.

The honest framing: these are *subjective, defensible* odds, recomputed each day as the tape changes, not a back-tested model. The discipline is that they must sum to 100% and each leg must be falsifiable.

3.4 The Burry tell & “today’s one chart that matters”

The Burry tell is deliberately *not* a trade. It’s the one structural observation nobody is looking at that resolves in ~6 months — e.g. “the consumer is spending on services while goods inventories pile up; this resolves badly.” It’s chosen by asking, across the sweep, *what divergence is building quietly that the day’s headlines ignore*, and it’s placed once, in “what the tape is missing,” as a standalone callout on Insights.

Today’s one chart that matters names the single instrument/level the market is actually trading around that session and the threshold that flips the story — “the level that changes the narrative.” It’s picked as the cleanest pressure point where the day’s dominant driver (L1) gets confirmed or broken.

3.5 The deep dives — where the opinions come from, and how they become *our* view

These are the LHS analysis blocks. For each, the pattern is the same: pull the public proxy, label it honestly, give the read *and* the trade implication. None of these claim a paid data feed — they're built from observable public signals plus Claude's synthesis.

Consensus bid / offer

This is literally: **state the single most consensus view in the market right now in one sentence, then the strongest argument against it in one sentence.** "Consensus" is judged from the sweep — what every wire, strategist note and price action is leaning on (e.g. "the ceasefire holds and oil normalises"). Our value-add is the offer side: the one clean reason it's wrong. We form it by asking which crowded view has the weakest load-bearing assumption.

Correlation regime

Flag the top 3–5 correlation *breaks* — across asset classes, sectors, and single names. For each: what broke, by how much, and *why*. The opinion: **a correlation breakdown means the dominant driver just changed** — so each break is read as evidence for or against the L1 regime call. We get the "by how much" from the live levels and the day's moves; the "why" is the synthesis.

Vol & skew

Public proxies, labelled as proxies (not a full surface — that needs a paid feed): VIX term structure (VIX9D / VIX / VIX3M — contango vs backwardation), the Cboe SKEW index, MOVE (rates vol), CVIX (FX vol) if available, put/call ratio, notable flow. Each gets a read *and* a trade implication, and the section ends with **one** options structure that fits today's vol regime — strikes quoted in deltas/percentages, never invented absolute prices.

Sector & RV

The 2 strongest and 2 weakest GICS sectors overnight, why, and whether the move is exhausted or has legs — plus one cross-sector or cross-region relative-value idea. The "exhausted vs legs" call leans on the same technical read used for trades (is the leader riding the upper Bollinger band / stretched above its MAs?).

Positioning & flows

Where the crowd is: CFTC Commitments-of-Traders net length for FX/commodities (weekly), fund-flow commentary, and the named **pain trade**. This section feeds the "positioning" component of every trade's conviction score (§5.1). The opinion is formed by combining the CoT/flow data with the absolute-RSI and Bollinger stretch — see the positioning definition in Part 5.

Funding & plumbing

SOFR, cross-currency basis, repo, bill issuance, reserves, RRP. One line when calm, **loud** when stressed. This is the Pozsar layer made literal — tracing whether today's move has a plumbing cause.

Catalyst calendar — how we screen it

Next 5 trading days. Per event: Day · Date · Event · Consensus · our one-line framework view · and the **asymmetry** (the trade if consensus is wrong, green/red). The screen is deliberately strict: *only genuine asymmetry makes the list*. An event where the outcome is fully priced and symmetric is dropped — we only keep events where being right on the non-consensus side pays materially more than being wrong costs. The earnings names are filtered separately (Part 4).

4 Earnings intelligence — what we screen, rank, and why

The universe filter (applied silently, before scanning)

A name only qualifies if it clears *all* of:

- **Market cap** $\geq$ \$10bn
- **Geography**: US (primary) or South Korea (secondary)
- **Sector**: Technology, Financials, Industrials, or Utilities only
- **Timing**: reports within 5 days pre or 3 days post

This filter is enforced in two places: `fetch_earnings.py` (the Finnhub fetcher maps Finnhub's industry strings to these sector buckets, checks the \$10bn cap and the `{US, KR}` geography, and writes only qualifying names into `earnings_data.md`), and again in the spec when Claude reads that file.

The data source order

1. Read `earnings_data.md` first — pre-fetched from Finnhub at 6am. Fields present here are tagged **SOURCED**.
2. Only where a field is marked "unavailable" do we supplement with targeted web search (implied move/straddle, short interest/float, post-earnings reaction), tagged **ESTIMATED**.
3. If the file is missing/empty, fall back entirely to web search and tag everything estimated.

The 4-pillar conviction score (0–2 each, max 8)

PILLAR	2 POINTS	1	0
Asymmetry	Implied move materially misprices the historical average, OR estimate dispersion unusually wide	Moderate signal	None / unavailable
Consensus (sell-side)	Majority buy, meaningful upside to PT, positive revisions	Mixed/neutral	Contradicts the idea / no coverage
Catalyst	Pre: identifiable catalyst consensus hasn't priced · Post: disproportionate reaction to print quality (oversold/overbought)	Plausible, not differentiated	No edge
Positioning	Short interest > 10% of float, OR extreme positioning	Notable lean	Neutral / unavailable

How the label (the ranking) is assigned — first match wins

- **HIGH CONVICTION** — score ≥ 6 , no pillar = 0, *all* pillars sourced.
- **HIGH — DATA GAP FLAGGED** — score ≥ 6 , no pillar = 0, but ≥ 1 pillar estimated/unverified.
- **MEDIUM** — score 4–5, OR any pillar = 0, OR consensus unverified.

- **LOW** — score <4 or ≥2 pillars = 0 → **excluded, not rendered**.

Two guardrails: (1) if the *consensus* pillar is unverified, conviction is capped at Medium no matter the score; (2) a High/High-gap label *requires* a one-sentence `conviction_rationale` explaining precisely why the asymmetry is real and attributable to the data — if it can't be written cleanly, the name is downgraded. Never fabricate an EPS, PT or implied-move figure; missing = "unverified." If consensus direction conflicts with the trade direction, the name carries `research_conflict: true`.

On the Earnings tab itself we show: the criteria (above), a one-line summary of today's ideas, *why these names* were picked out of the universe (the filter), and the sources — so the reader can audit the screen.

5 Trade Ideas & the Live Book

5.1 Conviction scoring — the /13

Every trade idea (and every live-book line) carries a conviction score with three stacked parts:

PART	COMPONENT	MAX	MEANING
Core /10	Gap: price vs fundamentals	3	How wide is the mispricing?
	Catalyst proximity	2	Is there a specific near-term event?
	Positioning extremity	2	Is the crowd leaning the wrong way?
	Cross-asset confirmation	2	Do other markets corroborate?
	Stop quality	1	Is the stop a clean level, not a vibe?
Technical /2	MAAs + Bollinger + Fibonacci	2	Does the chart agree, and is the entry not chasing an exhausted band?
RSI /1	Absolute 30/70	1	Is the position chasing an extreme? If so, flag and score 0.

So trade ideas and the live book are scored /13 (core 10 + TA 2 + RSI 1); the Derivatives Desk uses a base 8 + TA + RSI = /11. The TA and RSI parts are **recomputed live every refresh** from `fetch_rsi` and folded into the badge — they're not decorative.

5.2 Exactly how RSI, moving averages, Bollinger and Fibonacci are used — and where they come from

Where: all four are computed in `fetch_rsi.py` from ~1 year of daily closes pulled from Yahoo Finance (`query1.finance.yahoo.com/v8/finance/chart/<ticker>`). One year is chosen deliberately — it's the minimum that supports a 200-day MA and a 6-month Fibonacci swing. No library; the maths is hand-rolled stdlib.

RSI — 14-period Wilder, absolute convention

Computed with proper Wilder smoothing over 14 periods. The reading is the **classic absolute** one, and this is a hard rule:

- $RSI \geq 70$ = **overbought** (extended)
- $RSI \leq 30$ = **oversold** (washed out)
- 30–70 = **neutral**

Never average RSI against a rolling mean or measure standard-deviations-from-a-mean. That earlier approach washed the signal out and was wrong. A position is “crowded against us” when the tape is overbought and we’re long, or oversold and we’re short — that flags the trade and scores the RSI component 0.

Moving averages — 50 / 100 / 200-day

Used for **trend**: price above all three = uptrend; below all three = downtrend; otherwise mixed. The 50-vs-200 relationship gives the **golden cross** (50 above 200, confirms medium-term uptrend) or **death cross** (50 below 200). The percentage distance of price from each MA is also reported — that’s the “stretch” read used for positioning.

Bollinger Bands — 20-period, ± 2 SD

The key output is %B — where price sits between the lower band (0.0) and upper band (1.0). The rule for entry quality: a *long* wants %B in the lower half (≤ 0.55 , not chasing); a *short* wants the upper half (≥ 0.45). Riding the upper band ($\%B \geq 0.9$) = stretched; hugging the lower band (≤ 0.1) = washed out. **Bandwidth** (band width $\div$ mid) is the squeeze gauge — under ~8% means a low-vol coil about to expand (a crowded consensus about to break).

Fibonacci — retracements off the trailing 6-month swing

Take the high and low of the last ~126 sessions and lay 23.6% / 38.2% / 50% / 61.8% / 100% retracement levels across that range. The current price is matched to its nearest fib level. **These levels are where the stop and target are hung** — they’re the support/resistance the trade’s risk is defined against.

How they combine into the TA score (0–2)

- 2 = the trend agrees with the trade direction *and* the entry isn’t chasing an exhausted band.
- 1 = mixed (trend agrees OR entry is good, or trend is mixed).
- 0 = the chart contradicts the trade (trend against AND entry chasing).

Each trade’s dropdown spells out, in plain English, exactly what each tool is saying for that name on that day — that text is generated, not hand-written.

How “crowded” / positioning is judged (the four inputs)

1. Absolute-RSI (30/70).
2. MA stretch + Bollinger — far above the 50/100/200 and riding the upper band = extended/fade-prone; hugging the lower band below the MAs = sellers exhausted/squeeze-prone; a Bollinger squeeze = crowded low-vol consensus about to break.
3. CFTC CoT net length / fund flows / options skew.
4. The pain trade.

5.3 The RSI screener — universe and output

Every refresh, `run_screener()` scans a curated ~140-name cross-asset universe (hard-coded in `SCREEN_UNIVERSE`), computes 14-period RSI for each, and surfaces two columns on the Trade Ideas tab: **oversold** ($RSI \leq 30$, candidates to go higher) and **overbought** ($RSI \geq 70$, candidates to sell off), each with its full technical picture. The universe spans:



The list is editable in one place (`SCREEN_UNIVERSE` in `fetch_rsi.py`); there's a smaller legacy ~50-name list kept alongside for reference. Names that fail to fetch are counted as errors, not faked.

5.4 How a flagged idea gets into the book (and how that ties to “our book”)

The Trade Ideas tab generates four fresh ideas from today's data (rules: span ≥ 2 asset classes; ≥ 1 RV/spread; ≥ 1 options/vol idea; if there aren't four quality ideas, write fewer and say so — “forcing a trade is the trade”). The Live Book then shows a **selection summary**: which of today's flagged ideas go *into* the book and why, and why the others do *not*. The discipline: **everything in the book must trace to an idea in one of the ideas sections** — nothing appears in the book that wasn't argued for first.

The decision to implement is driven by the conviction stack: higher /13 scores, a clean catalyst inside the horizon, cross-asset confirmation, and a non-crowded RSI/positioning read get implemented; ideas that are right in thesis but chasing an extreme (RSI flagged) or lacking a clean stop level are left as “idea, not yet in the book.”

5.5 Duration, stops, and taking profit — how each is decided

Each trade in `trades.json` carries a full schema. The relevant fields and the logic:

FIELD	HOW IT'S SET
<code>horizon</code> ("days"/"weeks"/"months")	From catalyst distance and trade type . A <i>reactive</i> trade keys off a near event (days/weeks); a <i>pre-position</i> trade is structural (months) and carries a <code>min_hold_days</code> floor (≥ 30 , often 45) so it can't be shaken out intraday.
<code>stop</code>	A clean technical level, never a vibe — hung on a Fibonacci level or an MA/Bollinger edge from <code>fetch_rsi</code> . Stop quality is its own conviction point (1/10), so a trade with a fuzzy stop literally scores lower.
<code>target</code>	The next major support/resistance in the trade's direction (again a fib/MA level), giving a defined reward to size against the stop.
<code>min_hold_days</code>	Protects structural trades: never close a pre-position trade before this has elapsed, even if the stop looks hit intraday (e.g. the gold stop was touched on 10-Jun but held under the 45-day rule).
<code>conviction_breakdown</code>	The five core sub-scores, stored so the badge can be rebuilt and the call audited.
<code>history</code>	A dated, mark-by-mark log of level & P&L — this is what makes the book self-grading.

Closing logic, every refresh: if the live mark hits the stop → move to closed, result **STOPPED**. If it hits the target → closed, **TARGET**. If today's data *breaks the thesis* (independent of price) → close early, **DISCRETIONARY**, with a one-line reason. The scoreboard then reports total P&L%, hit rate, closed-trade count and best trade — so the book grades itself honestly over time rather than quietly forgetting losers.

6 Fable — the client portfolio

6.1 Who Fable is — read her like a story

On the website the client is **Fable** (the internal book file calls her “Aurora”). Here is her story, straight from

`client-book-json-Aurora.json`:

Fable is an **EMEA private-bank client of five years** (account opened June 2021), classified MiFID **Professional**. She’s **growth-oriented but with income needs**, and — crucially for the derivatives desk — **comfortable with structured products and derivative overlays**. Her base currency is the **euro**, but her book is **72% in US dollars** — a mismatch that matters every time the ECB moves. Total book $\approx$ **€50.2m**.

Her defining position is **Micron (MU)**: a small $\sim 2.5\%$ starter from 2021 that rode the HBM/AI-memory supercycle into a **10x gain** and is now **25.8% of the entire book** — a concentration problem — with earnings on 24 June and option premium running rich into the print. She bought **NVIDIA** near the January-2026 highs and is down $\sim 10\%$, caught in the rates-driven de-rate; **Broadcom**, bought into a late-May print, beat the estimate but missed the whisper and is -21% ; **LVMH** is -20% on soft luxury demand. Against those, **AMD** is a $+395\%$ winner she half-wants to trim, **SAP** a $+132\%$ European-software winner, and **TotalEnergies** ($+55\%$) and **Xetra-Gold** ($+109\%$) sit as her energy and tail hedges.

She took profit in Q1 and **never redeployed the cash**: **€2.4m** sitting idle in a current account and **\$3.0m** of trim proceeds earning zero while T-bills pay $4\%+$. She has a real-world liability the desk nets against: a **\$38,500/month Miami mortgage to 2031** — a natural short-USD position. Two of her bonds (a 2021 UST and a Siemens EUR IG) are underwater on *rates, not credit* — textbook bond-swap candidates, gated by CPI and the ECB.

The portfolio wasn’t generated randomly — it was constructed to be a *teaching book*: it deliberately contains a concentration problem (MU), a currency mismatch (EUR base / USD assets), winners to harvest income from (AMD, SAP), losers with different recovery routes (AVGO harvest, LVMH reverse-convertible, NVDA range-note), idle cash to deploy, and recognised hedges that should *not* be trimmed (gold, TTE). Every scanner rule has at least one position designed to trigger it (the `scanner_expected_triggers` tags on each holding).

6.2 The house-view engine — how we form a view, and rate it

For each holding Claude forms its own view and *shows its work*, stored in `house_views.json`. A view is one of **LIKE / NEUTRAL / LIKE-as-hedge / AVOID**, with a conviction **/10** built on a fixed rubric:

RUBRIC COMPONENT	MAX	WHAT EARNS IT
Evidence	3	≥2 recent, sourced, name-specific data points (analyst actions, prints, flows). Fewer → the view caps at NEUTRAL/estimated.
House alignment	3	Agreement with the JP Morgan GIS anchor + named authors + sell-side.
Regime fit	2	Consistency with today's brief regime (the live bear case).
Valuation & risk	2	The valuation/risk balance for the name.

The anchors that keep it honest:

- **JP Morgan GIS** (Global Investment Strategy) is the institutional house anchor every per-name view is checked against — mid-2026 read: overweight equities with a quality/AI-infrastructure tilt, neutral-to-short duration into sticky inflation, gold as a structural diversifier, cautious on consumer luxury.
- **Named author beats** (thematic, not quotes): Citrini Research (AI capex/power/semis), Doomberg (energy supply-side), Michael Howell (global liquidity & gold), Brent Donnelly (FX/positioning).
- **Consistency lock to the brief.** The views are locked to the morning brief's regime — the scanner *never contradicts the brief*. E.g. MU stays LIKE even though the brief's live bear case is an AI de-rating, because hedging a 26% concentration is position management, not a view reversal.
- **Staleness:** the whole view set has a `stale_after` date; views are re-formed, not assumed perpetual.

How “like / neutral / hate / hate-as-a-hedge” is decided

- **LIKE** — franchise intact + house/regime support + a defined add level. Unlocks the add-intent shelf (e.g. NVDA: range note or a cash-secured put at 195). But a LIKE on a *rich* winner means *overwrite for income, not chase* (SAP).
- **NEUTRAL** — no near-term catalyst either way; opens income / staged-exit structures (overwrite, decumulator) but *not* accumulation (AMD after +395%; AVGO post-whisper-miss → harvest lane).
- **AVOID** — thesis broken; converts to a clean harvest-and-rotate. (LVMH sits at NEUTRAL, one demand downgrade from AVOID.)
- **LIKE-as-hedge** — held as *insurance*, so the normal “trim the winner” rule is **suppressed**. TotalEnergies (+55%) and gold (+109%) both trip the winner-trim rule on the raw numbers, but brief-consistency overrides it: “you don’t sell the fire insurance while the building is still smoking.” Gold is the *highest*-conviction view (8/10) precisely because it’s the lowest-conviction *directional* bet — it wins on both of the brief’s tails (a rate-cut path and a geopolitical re-escalation).

6.3 The Portfolio tab’s charts and the IV calculation

The Portfolio tab renders three allocation **pie charts** computed in `book_scanner.compute_allocations()`: by **asset class** (whole book incl. cash), by **equity sector** (equities only), and by **region** (by listing currency; gold ETC = Global). Plus the holdings table with live marks and the house-view engine output.

The **implied-vol explainer** above the table is real maths, not a label. `fetch_ivol.py`:

- For US single names with listed options (MU, NVDA, AVGO, AMD, SPY): it pulls the option chain from Yahoo (stdlib cookie+crumb handshake), takes the ~360-day ATM contract, and **solves for implied vol itself by inverting Black–Scholes** on the bid/ask mid (bisection, 80 iterations) — because Yahoo’s own `impliedVolatility` field returns ~0 and can’t be trusted. It also computes the 90% put skew.
- For EU names with no Yahoo options (LVMH, SAP, TTE) and the gold ETC: it computes **realized vol** from a year of daily closes (63-day window, annualised) as an implied-vol proxy, with a percentile ranked against the trailing-year rolling distribution.
- The IV *percentile* stays “estimated” until the history file accumulates ~60 daily snapshots, after which it’s “sourced.” The snapshot is appended to `ivol_history.json` daily.

This matters because the derivatives ideas lean on rich/cheap vol — and the desk wants a *computed, sourced* vol number, not a guess.

7 The Derivatives Desk

The desk turns the portfolio scan into a ranked shelf of structured-product and derivative ideas, each tagged by **product type + objective**, grouped by asset class. It's built in `book_scanner.py`.

The goal of the desk for this book

The one-screen action plan (`_overall_summary`) states the objective every idea must serve: **de-risk the concentration, put the idle cash to work, keep the hedges — and add AI exposure through software and power, not more semiconductors**. Every idea is consistency-locked to the brief and to JP GIS; nothing contradicts the macro view or the per-name house views.

The shelf (what trades cleanly, by asset class)

ASSET CLASS	INCOME / PARTICIPATION	HEDGING / OTHER
Equities	RevCon, FCN, Phoenix, digital review, autocall Market Plus, BREN, protected participation	Equity OTC, full creative latitude: accumulators/decumulators, call/put spreads, risk reversals, prepaid variable forwards, total-return swaps, worst-of/dispersion, variance/vol
FX	DCS (dual-currency / deposit) for income	Forwards, seagulls, puts, accumulators for hedging
Rates	Range accruals only	

The shelf is a guideline, not a cage. If an idea genuinely needs a structure off-shelf (or off-piste OTC, where almost anything is possible), the desk still suggests it — and tags it “please check with your derivs team on feasibility.” The rule is: never hold back a good idea because it's off-shelf; just flag the feasibility check.

How an idea is judged and ranked

Each idea (`_idea(...)`) carries a conviction sub-score across four keys — **setup, pricing, catalyst, fit** — each 0–2 with a *confidence* tag (sourced/estimated), plus the live TA + absolute-RSI for its underlier. Ideas are sorted into three tiers:

- FIRE** — sourced, high-fit, ready to execute (e.g. the MU collar + SBL + post-print decumulator; the Oracle capital-protected note; the EUR 2y steepener note).
- WATCH** — good thesis but a key input (basket, IV) not yet sourced (e.g. the AI-power thematic note).
- SUPPRESS** — conflicts with another idea or the house view (e.g. a DCS that would fight the FX hedge; trimming a recognised hedge).

Fit is the heaviest discriminator: an idea scores high only if it matches Fable’s actual pattern (AI/semis-heavy, software, large-cap quality, energy+gold hedges, US duration, structure-comfortable) *and* ideally *reduces* the concentration. That’s why “add AI” here means Oracle (cloud) and Adobe (software) and a power basket — same conviction, a different leg of the build-out — rather than more silicon. Tenor floors are enforced: equity structured products $\geq 3\text{m}$, OTC $\geq 1\text{m}$, rates structured products $\geq 2\text{y}$.

Ideas specific to *this* book — worked examples

- **The concentration (MU, 25.8%)**: a zero-cost collar (rich calls finance fat puts into the 24-Jun print) + a Lombard/SBL line against the collared stock, then — *after* the binary print — a decumulator sized $\leq 50\%$ to exit above market. The decumulator is *timed, not killed*: decumulating with 2x gearing through a catalyst risks being forced to sell double into a +20% gap. An OTC **prepaid variable forward** on ~a third of the position is offered as the alternative.
- **The FX mismatch (72% USD on a EUR base)**: a T-bill ladder + FX forward strip, and a seagull on the residual after the ECB — sized off the *liability-netted* exposure (the Miami mortgage is a natural short-USD that nets down the hedge).
- **The idle cash**: EUR deposit campaign + range accrual; USD T-bill ladder + a cash-secured NVDA put at 195 (get paid to wait at the level she’d add anyway).

Recovering losses — the explicit route per underwater name

`_loss_recovery()` gives each loser a precise “how,” plus a faster structured-product/OTC-at-margin “accelerator” where it genuinely fits:

POSITION	THE ROUTE	THE ACCELERATOR
AVGO (–21%)	Harvest the loss now (post-print IV crushed → tight replacement spreads) to bank tax against the eventual MU exit; re-enter the AI-ASIC thesis via a 12–18m buffered note; in ~30 days sell a reverse convertible once vol rebuilds.	Run the buffered note + an OTC accumulator at margin (discount average-down) to claw back the 21% faster — margin capped so a full obligation is always deliverable.
LVMH (–20%)	Sell a 6-month reverse convertible struck at the add level; steep luxury put skew fattens the coupon — paid to wait, and if assigned she owns more at a discount. FCN if the coupon must be certain.	Pair the RevCon coupon with a short OTC accumulator at margin to leg into the discount on a daily clip.
NVDA (–10%)	Sell a BREN / range note on the 195–235 consolidation — paid a coupon while it holds the range (paper loss → paid waiting).	Add a cash-secured put at 195 (not simultaneously with the BREN, per delta-netting) — premium offsets the drawdown and commits the add at support.
UST '31 + Siemens '30	Switch both into current-coupon paper of the same credit — the loss is already taken; the switch banks it and ~triples the carry. CPI gates the UST leg; the ECB gates the EUR leg.	Not a margin/OTC candidate — the recovery <i>is</i> the carry pickup, banked cleanly via the switch.

8 Reference — file map

FILE	ROLE
<code>PROMPT.md</code>	The operating spec — the strategist system prompt, the 24h sweep rules, the voice DNA, the conviction rubric, every section, the <code>brief</code> schema. Claude reads this on “run the Market Map.”
<code>gen_2026_MM_DD.py</code>	The dated generator — today’s authored <code>brief</code> dict + the orchestration that fetches live data and renders. A new one each day.
<code>live_levels.py</code>	TradingView scanner client — live macro levels + book marks (the “TradingView feed”).
<code>fetch_rsi.py</code>	RSI (absolute 30/70), 50/100/200 MAs, Bollinger, Fibonacci, valuation multiples, and the ~140-name screener — all from Yahoo Finance.
<code>fetch_ivol.py</code>	ATM implied vol by Black–Scholes inversion on Yahoo option marks; realized-vol proxy for names without listed options. Writes <code>ivol_history.json</code> .
<code>fetch_earnings.py</code>	Finnhub earnings fetcher — runs as a GitHub Action at 6am, writes <code>earnings_data.md</code> .
<code>book_scanner.py</code>	Client-book engine — live marks, metrics, house-view attach, allocation pies, the ranked derivatives shelf, and loss-recovery routes.
<code>book.py</code> · <code>charts.py</code> · <code>shark_format.py</code> · <code>earnings.py</code>	The renderers — trade-book persistence/scoreboard, inline-SVG charts, the six-tab Shark Tank formatter, and the earnings section.
<code>client-book-json-Aurora.json</code>	Fable’s portfolio (positions, cash, liabilities, summary).
<code>house_views.json</code>	Per-name views with conviction breakdowns, evidence, sources, brief-consistency notes.
<code>trades.json</code> · <code>regime_log.json</code>	The self-grading trade book (open/closed + dated history) and the regime-banner log.
<code>sources.py</code>	The trusted-source allowlist for the narrative.
<code>vercel.json</code> · <code>netlify.toml</code> · <code>.vercelignore</code>	Static-site deploy config (no build step; Python excluded).
<code>.github/workflows/</code>	<code>earnings-data.yml</code> (6am Finnhub) and <code>fallback-refresh.yml</code> (07:30 UTC data-only safety net).

`index/insights/earnings/trades/portfolio/ideas.html`

The six rendered tabs served by Vercel.

9 Glossary

Terminal / shell	The text window for typing commands (here, PowerShell).
Claude Code	Claude with the ability to read/write files, run commands, search the web, in the terminal.
Git / repository	A folder whose change history is tracked; “commit” = save a snapshot, “push” = upload it to GitHub.
GitHub	Cloud host for the repo + runner of scheduled Actions.
Vercel / Netlify	Static hosting that republishes the HTML whenever GitHub updates.
MCP	Model Context Protocol — a standard adapter for plugging tools/data into Claude. (Not used in the live data path here.)
RSI (absolute)	14-period momentum gauge; ≥ 70 overbought, ≤ 30 oversold. Never averaged vs a mean.
Bollinger %B	Where price sits between the bands (0 = lower, 1 = upper); the entry-stretch read.
Golden / death cross	50-day MA crossing above / below the 200-day MA.
Fibonacci retracement	23.6–61.8% levels across a swing high–low; where stops/targets are hung.
Implied vs realized vol	Implied = solved from option prices (Black–Scholes inversion); realized = measured from past returns (the proxy when no options exist).
Collar / RevCon / FCN / BREN / decumulator / seagull	Structured/OTC payoffs — see the desk shelf table (§7).
Pre-position vs reactive trade	Structural (months, min-hold floor) vs tactical (days/weeks, off a near event).

10 Honesty & limits — read this

- **The narrative is authored, the numbers are live.** Marks, RSI, technicals, the screener, the book and vol are fetched real at run time. The macro story is only as fresh as the last genuine web sweep — and the UI says which (the fallback banner + coverage bar).
- **No “TradingView MCP.”** Live levels come from TradingView’s public scanner endpoint called directly by `live_levels.py`; it’s unofficial and can change, with file-mark fallback.
- **Vol & skew are public proxies**, labelled as such — not a full options surface (that needs a paid feed).
- **Bonds keep yield-derived marks** (no clean free bond price feed); some EU equity marks are indicative until the live refresh resolves.
- **Bull/Base/Bear probabilities are disciplined judgement, not a back-tested model** — they must sum to 100% and each leg must be falsifiable.
- **Fable is a demo book.** House-view author references are thematic characterisations, not quotes or real calls. JP GIS framing is illustrative. *Not investment advice.*

This 101 was assembled by reading the project’s own source — `PROMPT.md`, the generators, `live_levels.py`, `fetch_rsi.py`, `fetch_ivol.py`, `fetch_earnings.py`, `book_scanner.py`, `house_views.json`, `client-book-json-Aurora.json`, `trades.json`, the deploy config and the GitHub workflows — so every claim above traces to code that runs. Repo: `harryimposter/shark-tank`.